

Relatorio Tecnico - API REST de Gerenciamento Academico

Atividade Prática: Desenvolvimento de Projeto Utilizando Bibliotecas Python

1. Objetivo do Sistema

O projeto desenvolvido consiste em uma API REST para gerenciamento academico, implementada integralmente em Python. O sistema permite cadastrar, consultar, atualizar e remover alunos, cadastrar disciplinas, realizar matriculas em disciplinas e gerar um relatorio academico com indicadores como media geral, quantidade de alunos aprovados, em recuperacao e reprovados.

2. Biblioteca Utilizada

A principal biblioteca externa utilizada foi o Flask, um microframework Python voltado ao desenvolvimento de aplicacoes web e APIs REST. O Flask foi aplicado para criar rotas HTTP, receber dados em formato JSON, processar requisicoes e retornar respostas padronizadas tambem em JSON.

Tambem foram utilizados recursos da biblioteca padrao do Python, como datetime para registrar a data de matricula e statistics para calcular medias academicas.

3. Principais Funcionalidades

- Listagem de alunos cadastrados.
- Cadastro de novos alunos com validacao de campos obrigatorios.
- Consulta individual de aluno pela matricula.
- Atualizacao de dados do aluno, incluindo notas.
- Remocao de aluno e de seus vinculos com disciplinas.
- Listagem e cadastro de disciplinas.
- Matricula de aluno em disciplina.
- Geracao de relatorio academico com totalizadores e situacao dos alunos.

4. Entrada, Processamento e Saida de Dados

A entrada de dados ocorre por meio de requisicoes HTTP enviadas para a API, principalmente utilizando o formato JSON. O processamento envolve validacao dos campos recebidos, verificacao de duplicidade, calculo de medias e classificacao da situacao academica do aluno. A saida e retornada em JSON, contendo mensagem de status, dados processados e codigos HTTP adequados.

5. Rotas da API

Metodo	Rota	Finalidade
GET	/	Verificar funcionamento da API e listar rotas disponiveis.
GET	/alunos	Listar alunos com media e situacao.
POST	/alunos	Cadastrar um novo aluno.
GET	/alunos/<matricula>	Buscar um aluno especifico.
PUT	/alunos/<matricula>	Atualizar os dados de um aluno.
DELETE	/alunos/<matricula>	Remover um aluno.
GET	/disciplinas	Listar disciplinas cadastradas.
POST	/disciplinas	Cadastrar uma nova disciplina.
POST	/matriculas	Matricular aluno em disciplina.
GET	/relatorio	Gerar relatorio academico consolidado.

6. Exemplos de Funcionamento

Executar o sistema:

```
python app.py
```

Exemplo de cadastro de aluno via JSON:

```
{ "matricula": "2026003", "nome": "Mariana Alves", "curso":  
"Engenharia de Software", "email": "mariana@email.com",  
"notas": [9, 8.5, 10] }
```

Ao acessar GET /relatorio, a API retorna indicadores processados, como total de alunos, total de disciplinas, media geral e quantidade de alunos por situacao.

7. Boas Praticas Aplicadas

- Codigo organizado em funcoes com responsabilidades definidas.
- Rotas separadas por recurso da API.
- Validacao de dados antes do cadastro ou atualizacao.
- Respostas JSON padronizadas.
- Uso de codigos HTTP adequados para sucesso, erro de validacao, conflito e nao encontrado.
- Comentarios explicativos em pontos importantes do codigo.

8. Conclusao

O desenvolvimento da API REST demonstrou a aplicacao pratica de bibliotecas Python no contexto de desenvolvimento de sistemas. O Flask permitiu estruturar uma solucao funcional, objetiva e escalavel, com entrada, processamento e saida de dados. A atividade tambem reforcou conceitos importantes como organizacao de codigo, validacao de informacoes, padronizacao de respostas e criacao de rotas para integracao entre sistemas.

Arquivo principal do projeto: app.py

Documento gerado para entrega da atividade pratica.

```
{
  "dados": {
    "biblioteca": "Flask",
    "rotas": [
      "GET /alunos",
      "POST /alunos",
      "GET /alunos/<matricula>",
      "PUT /alunos/<matricula>",
      "DELETE /alunos/<matricula>",
      "GET /disciplinas",
      "POST /disciplinas",
      "POST /matriculas",
      "GET /relatorio"
    ],
    "sistema": "API REST de Gerenciamento Academico"
  },
  "mensagem": "API em funcionamento"
}
```

Para rodar no seu navegador use <http://127.0.0.1:5000>

```
{
  "dados": [
    {
      "curso": "Análise e Desenvolvimento de Sistemas",
      "email": "ana.souza@email.com",
      "matricula": "2026001",
      "media": 8.43,
      "nome": "Ana Souza",
      "notas": [
        8.5,
        9.0,
        7.8
      ],
      "situacao": "Aprovado"
    },
    {
      "curso": "Sistemas de Informação",
      "email": "carlos.lima@email.com",
      "matricula": "2026002",
      "media": 7.17,
      "nome": "Carlos Lima",
      "notas": [
        6.0,
        7.5,
        8.0
      ],
      "situacao": "Aprovado"
    }
  ],
  "mensagem": "Operação realizada com sucesso"
}
```

```
{
  "dados": [
    {
      "carga_horaria": 80,
      "codigo": "PY101",
      "nome": "Programacao Python"
    },
    {
      "carga_horaria": 60,
      "codigo": "BD201",
      "nome": "Banco de Dados"
    }
  ],
  "mensagem": "Operacao realizada com sucesso"
}
```